HYPERSTEER: Activation Steering at Scale with Hypernetworks

Jiuding Sun*
Stanford University
sunjd24@stanford.edu

Sidharth Baskaran*
Pr(Ai)^{2}R Group
Georgia Institute of Technology
sidnbaskaran@gmail.com

Zhengxuan Wu
Stanford University
zhengxuan@stanford.edu

Michael Sklar
Confirm Labs
michaelbsklar@gmail.com

Christopher Potts
Stanford University
cgpotts@stanford.edu

Atticus Geiger
Pr(Ai)^{2}R Group
atticusg@gmail.com

arXiv:2506.03292v1 [cs.CL] 3 Jun 2025

Abstract

Steering language models (LMs) by modifying internal activations is a popular approach for controlling text generation. Unsupervised dictionary learning methods, e.g., sparse autoencoders, can be scaled to produce many steering vectors, but lack guarantees on the individual efficacy of each vector and control over the coverage of relevant steering tasks. In contrast, supervised methods for constructing steering vectors are targeted and effective, but require more data collection and training for each additional steering vector produced. In this work, we introduce HYPERSTEER, a family of hypernetwork-based architectures which are trained end-to-end to generate steering vectors conditioned on the natural language steering prompts and the internals of the steered LM. In our evaluations, we show that scaling HYPERSTEER with thousands of steering prompts exceeds the performance of state-of-the-art activation steering methods, even on steering prompts never seen during training. Moreover, HYPERSTEER performs on par with steering-via-prompting. We release ours at stanfordnlp/axbench.

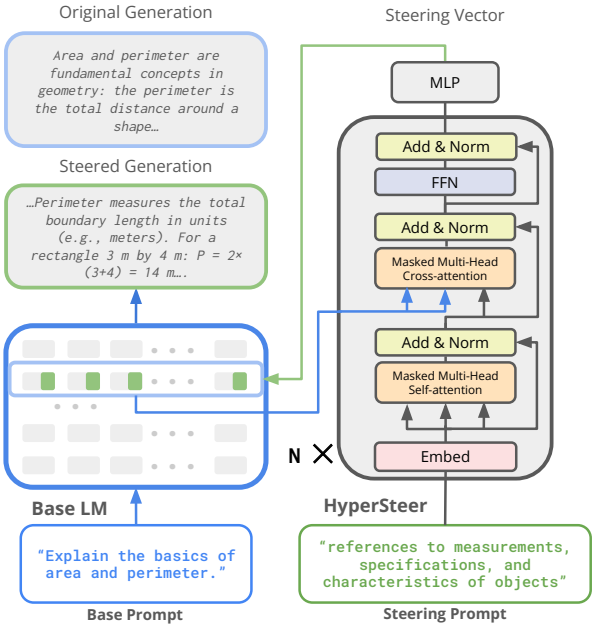

Figure 1: The state-of-the-art HYPERSTEER Model: A transformer hypernetwork uses self attention to process a steering prompt and uses a cross attention module to read from the residual stream of a base LM run on a second prompt. The hypernetwork outputs a steering vector that is added to the base LM residual stream.

1 Introduction

How can the outputs of a language model (LM) be reliably controlled? With instruction-tuned LMs, the standard approach is prompt engineering. However, prompting-based approaches face challenges from user jailbreaks, forgotten instructions, and robustness to model misalignment. A more aggressive approach is (parameter-efficient) fine-tuning, but this requires modifying or injecting new parameters into the model. Activation steering (Giulianelli et al., 2018) occupies a middle ground: it is lightweight to implement (no parameters are modified or added) and can affect the model in ways standard prompting cannot.

Methods for obtaining steering vectors can generally be classified into two groups: *unsupervised*, e.g., Sparse Autoencoders (SAEs) (Hernandez et al., 2022; Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Marks et al., 2025), which produce a large number of unlabeled steering vectors, and *supervised*, where task-specific steering vectors are either directly trained (Wu et al., 2024a, 2025) or derived using labeled data (Li et al., 2023a; Marks and Tegmark, 2023; Turner et al., 2023; Rimsky et al., 2024). Unsupervised methods are scalable, but lack the ability to create steering vectors for specific tasks. Alternatively, a supervised approach is effective and targeted, but requires per-task data collection and/or training.

Our main contribution is HYPERSTEER, a suite of end-to-end trainable hypernetwork architectures which learn to generate steering vectors for an

*Equal contribution.

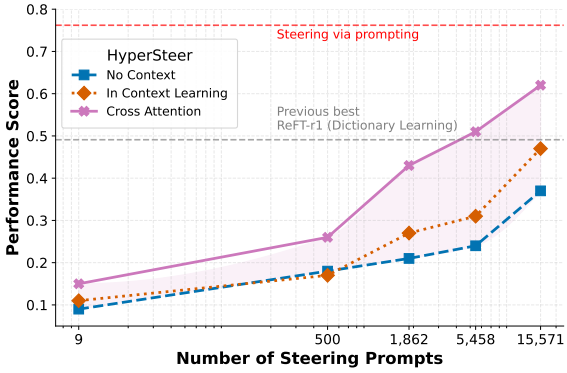

Figure 2: Performance on steering prompts that have **never been seen during training** for HYPERSTEER variants as the number of steering prompts used in training increases (x-axis in log scale). An exponential increase in training data results in an approximately linear increase in performance. When trained on all ≈16k steering prompts in AxBench, the performance of HYPERSTEER on steering prompts never seen during training surpasses `ReFT-r1` steering vectors which are trained and evaluated on the same steering prompt.

Method	Gemma-2-2b Held-out	Gemma-2-2b Held-in	Gemma-2-9b Held-out	Gemma-2-9b Held-in
Prompting	**0.762**	0.731	**1.091**	1.075
Fine-tuning				
LoReFT	–	0.722	–	0.777
SFT	–	0.714	–	–
LoRA	–	0.641	–	0.602
Activation Steering				
ReFT-r1	–	0.509	–	0.630
DiffMean	–	0.178	–	0.322
SAE	–	0.151	–	0.191
SAE-A	–	0.132	–	0.186
HYPERSTEER				
– No Context	0.373	0.512	0.633	0.751
– In Context Learning	0.480	0.547	0.760	0.842
– Cross Attention	0.608	**0.742**	0.934	**1.091**

Table 1: Steering results for baseline methods from AxBench and our three HYPERSTEER variants, evaluated on `Concept500-HO` (steering prompts never seen during training) and `Concept500-HI` (steering prompts seen during training, base prompts unseen during training). The cross attention variant outperforms all other variants by a large margin on held-out evaluation and approaches prompting performance on held-in evaluation, while all variants outperform the `ReFT-r1` baseline on held-in evaluation. The intervention happened at Layer 20 of both models.

instruction-tuned base LM, conditioned on the steering prompt and (optionally) the base LM prompt and internal activations. We train and evaluate on the steering prompts from AXBENCH (Wu et al., 2025). Our best HYPERSTEER variant outperforms the previous state-of-the-art activation steering method from AXBENCH, even on held-out steering prompts, never seen during training. We show steering performance scales logarithmically with number of steering prompts during training. Surprisingly, we are even able to match the performance of the steering-via-prompting, which outperformed all activation steering methods on AxBench. In sum, HYPERSTEER combines the scalability of unsupervised dictionary learning with the targeted control of supervised activation steering.

2 Preliminaries

Activation Steering The goal of activation steering is to elicit a certain behavior from a base LM $\mathcal{B}$ run on an base prompt by adding a steering vector to a hidden vector $\mathbf{h}$. Our hypernetwork-based approach to activation steering has the more general goal of taking any base prompt x, e.g., *Explain how to sort lists of numbers.*, and any steering prompt s, e.g., *Output using C/C++ programming syntax*, and producing a steering vector Δ_s^x added to $\mathbf{h}$. Denote the steered output:

$$\hat{\mathbf{y}}_{\text{steer}} = \mathcal{B}\big(x \mid \mathbf{h} \leftarrow \mathbf{h} + \Delta_s^x\big). \tag{1}$$

Hypernetworks Prior work has shown hypernetworks to be effective at zero-shot adaptation of language models on a variety of tasks, matching or exceeding comparable methods such as fine-tuning with parameter efficient methods (Phang et al., 2023). Formally, a hypernetwork (Ha et al., 2016) is a function $\mathcal{H} : \mathcal{T} \to \Theta$ that maps tasks $t \in \mathcal{T}$ to parameters $\theta_t = \mathcal{H}(t)$.

For our purposes, the tasks are pairs on input and steering prompts $(x, s) \in \mathcal{T}$ and the output parameters are steering vectors:

$$\mathcal{H}(x, s) = \Delta_s^x \in \mathbb{R}^d. \tag{2}$$

Dataset and Evaluation For all experiments, we use AXBENCH (Wu et al., 2025) as the training dataset and evaluation harness. The training data sets consist of a total of 16,000 steering prompts sourced from `GemmaScope` Sparse Autoencoder (SAE) feature labels (Lieberum et al., 2024) and base prompts sourced from a diverse instruction pool (see App. A.5 for details). We keep fixed ratios of base prompt to steering prompt (72 : 1 for train; 10 : 1 for evaluation).

For evaluation, the steering prompts are applied on a set of different base prompts sourced from AlpacaEval (Li et al., 2023b), and a `gpt-4o-mini` judge model (OpenAI et al., 2024) computes discrete scores in $\{0, 1, 2\}$ along three dimensions of

success: following the base prompt, following the steering prompt, and text fluency. The harmonic mean of these three scores is the final metric, which we refer to as the *steering performance*.

We evaluate on two datasets: `Concept500-HI` (held-in), the standard AxBench setting with 500 steering prompts seen during training plus unseen base prompts, and `Concept500-HO` (held-out), a test set with 500 steering prompts not seen during training. Notably, the only the prompt engineering baseline from AxBench can be evaluated on the held out test set alongside HyperSteer.

Baselines We report the fine-tuning, activation steering, and prompting baselines from AxBench. The state-of-the-art activation steering method `ReFT-r1` is an important point of comparison for our method. See Appendix A.6 for details.

3 HyperSteer

We consider multiple HyperSteer variants, all of which employ a transformer hypernetwork $\mathcal{H}$ with L layers and residual stream representations $\mathbf{h}_l^t$ for each layer l and token t from the steering prompt $\mathbf{s}$ with T tokens. All variants have an MLP module that maps from the residual stream of the last layer and token to a steering vector:

$$\mathcal{H}(s, x) = \mathsf{MLP}(h_L^T) = \Delta_s^x. \quad (3)$$

We train $\mathcal{B}$ on a language modeling loss using the output $\hat{\mathbf{y}}_{\mathsf{steer}}$ (see Equation 1) of the base model $\mathcal{B}$ under a steering intervention and an expected output $\mathbf{y}_{\mathsf{label}}$ from AxBench:

$$\mathcal{L}_{\mathsf{LM}}(x, s) = \mathsf{CrossEntropy}(\hat{\mathbf{y}}_{\mathsf{steer}}, \mathbf{y}_{\mathsf{label}}). \quad (4)$$

We consider a range of architectures with incrementally more access to the base LM $\mathcal{L}$ and prompt x.

No context No access to the base prompt x or the base LM $\mathcal{B}$, meaning $\mathcal{H}(x, s) = \mathcal{H}(s) = \Delta_s$.

In Context Learning This variant appends the base prompt x to the source prompt s and feeds the resulting text into hypernetwork $\mathcal{H}$.

Cross Attention Our best-performing variant conditions on both the steering prompt s and the internal activations of the base LM $\mathcal{B}$ run on prompt x. A cross-attention modules at each layer of the hypernetwork $\mathcal{H}$ uses the hypernetwork residual stream for attention head queries and outputs and the base LM residual stream at the steering layer l as attention head keys and values (App. A.3). This is a simplification of HyperDAS (Sun et al., 2025).

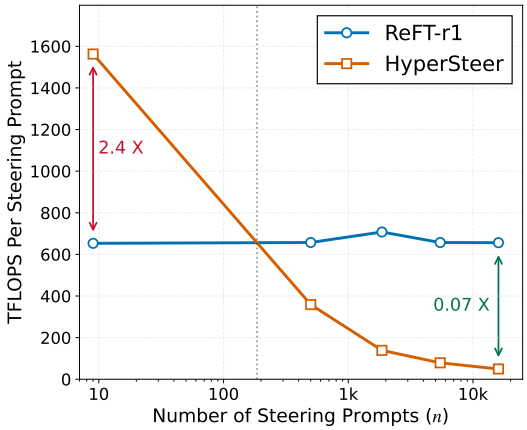

Figure 3: As the number of steering prompts in our training dataset increases, the teraFLOPs (TFLOPS) required to attain a similar loss on a *held-in* evaluation set (steering prompts seen during training, but base prompt unseen during training) decreases for our best HyperSteer variant (cross attention). This value is approximately constant for our dictionary learning baseline of `ReFT-r1`. See Appendix A.3.2 for details.

4 Experiments

We implement HyperSteer building on the AxBench (Wu et al., 2025) codebase using `pyvene` (Wu et al., 2024b) to implement steering methods. Training runs are done on a single NVIDIA `A100-80GB` GPU. Hyperparameter details in A.2. Ourexperiments steer two base LMs, namely, the `2B` and `9B` variants of `Gemma-2` instruction-tuned. (Rivière et al., 2024), with its parameters frozen during training. The hypernetworks used are unfrozen copies of the base model with later layers removed. We train each HyperSteer variant on steering prompt datasets ranging from 10 up to the full $\approx$ 16000 steering prompts available in AxBench.

Generalization Results We evaluate HyperSteer on AxBench's `Concept500-HI` and `Concept500-HO` and report results in Table 1. **our cross-attention HyperSteer variant performs better on unseen steering prompts than every supervised activation steering baseline trained and evaluated on the same steering prompt.** However, HyperSteer falls slightly behind prompting and the best fine-tuning baselines. Figure 2 shows that the cross-attention variant outperforms the other architectures at every dataset scale.

Compute efficiency We study the efficacy of HyperSteer with respect to the FLOPs needed to maintain the evaluation loss on a held-in dataset